Nock Compilation (Subject Knowledge Analysis II)

K. Afonin ~dozreg-toplud
Urbit Foundation

Abstract

The design choices for the Nock compilation pipeline, which has its roots in the Subject Knowledge Analysis (SKA) work by Edward Amsden ~ritpub-sipsyl and Joe Bryan ~master-morzod, are presented here. In particular, I describe a call graph construction algorithm that greatly improves on the previous approach, and describe the shared data flow analysis and code generation step that produces linear static single-assignment code from a call graph of SKA-functions with tree-shaped Nock code.

Contents

Urbit Systems Technical Journal III:1 (2026): 43–67.

Address author correspondence to ~dozreg-toplud.

1 Introduction

Nock, unlike conventional languages, does not have a notion of a "code object", or a "function", or any other construct that corresponds to known callable code. The Nock 2 formula `[2 b c]`—and Nock 9 by extension, as it is just a macro over Nock 2—is the equivalent of `eval` in other languages and reduces like this (`~sorreg-namtyv`, 2013):

```
*[a 2 b c]          *[*[a b] *[a c]]
```

That is, one evaluates `c` against the original subject `a` to obtain a formula, then reduce that formula with `*[a b]` as our new subject. Nock is expressive enough that `*[a c]` can be unknowable in the general case without actually running the code.

But while it is unknowable in the general case, in practice we can almost always know in advance what formula will be evaluated. That is because in practice the formula-formula `c` is almost always:

- A Nock 0, with the formula being pulled from the known subject (e.g. desugaring of Nock 9); or
- A Nock 1, with the formula being a constant/quoted value (i.e. `|-` loops, where the formula does not come from the subject but is instead quoted into the outer formula).

This fact allows us to introduce the notion of a *SKA-function* object, which is identified by a Nock formula and a masked subject. SKA stands for Subject Knowledge Analysis, the name of the call graph construction algorithm for Nock developed by Edward Amsden `~ritpub-sipsyl` and Joe Bryan `~master-morzod`.[1] In this paper, SKA (in the spirit of notation abuse) will mean both the call graph construction algorithm for Nock and the overall compilation pipeline. The mask includes only the code that could be used by the SKA-function, either by itself or transitively by its callees—that is, the subset of the subject's noun structure that the function and its callees might actually reference. A SKA-function can use any Nock operations, including raw Nock 2 when `*[a c]` could not be deduced (an *indirect* Nock call), but it can also call other SKA-functions.

Once the function call graph is obtained, the next step is to discover which parts of the subject are actually used as data by each function. Without it, each function would have to have a signature `(subject: noun -> noun)`, which would lead to unnecessary busywork when it comes to function calls—the entire subject of a callee would have to be consed up, for it to be deconstructed later by the callee.

With that step done, the actual code generation for a SKA-function can be performed on-demand, avoiding extra work for functions with total jets and functions that are never called.

2 Call graph construction

2.1 Previous work and motivation for redesign

Multiple algorithms were developed by Edward Amsden `~ritpub-sipsyl` to construct the call graph from a subject-formula pair.[2] I took his latest implementation and constructed a similar algorithm (`~dozreg-toplud`, 2026), whose primary characteristic was execution-order (i.e. depth-first)

[1] See `~dozreg-toplud`, "Subject Knowledge Analysis," *Urbit Systems Technical Journal* volume 3 issue 1, pp. 43–67.

[2] See `~ritpub-sipsyl` and `~master-morzod`, "Subject Knowledge Analysis", Urbit Lake Summit, ~2024.6.19.

inference and traversal of the call graph. The traversal very closely resembled Tarjan's strongly connected component (SCC) algorithm—where an SCC is a maximal set of mutually-reachable nodes in the call graph—except the graph was inferred at the same time as it was traversed, and the assumptions made in an SCC were validated upon returning from that SCC. If any assumption was invalid, then the entire SCC was reanalyzed with the assumption added into an exclusion list.

As the implementation matured and was tested on more and more complex workloads, it became apparent that the validation and reanalysis approach was too costly: analysis time increased exponentially with the depth of the SCC stack. Independently of this problem, I examined the literature to figure out another fixed-point algorithm. This led me to the realization that the call graph itself can be expressed as a fixed point of a partial evaluation function.

2.2 Formal Design

Before the formal design, let me introduce two pieces of vocabulary. A *partial evaluator* runs a formula as far as it can using only the information known statically, producing an unknown result otherwise. A *fixed point* of a function is an input that the function maps to itself: we apply our analysis repeatedly until the result stops changing, and that stable result is the fixed point we are after.

First, let me define the domain of the function whose fixed point we are going to find. It is going to be a mapping $\mathbb{G} : \text{IDENTITY} \to \text{DATUM}$, where:

- IDENTITY is a pair $(\textbf{more} : \texttt{Sock}, \textbf{formula} : \texttt{Cell})$, where `Sock` is a partial noun (a noun with some structure known and some left as unknown holes), **more** represents the entirety of the subject with which the eval was performed, and **formula** is a Nock formula that is being evaluated in this SKA-function call.

- DATUM is the information necessary for the partial evaluation of the callers of the SKA-function with the given

IDENTITY, which is at least: **less** : `Sock` for a minimized subject, that includes only the parts that are used for eval by that SKA-function and its transitive callees; **product** : `Sock` for the product of the SKA-function call; and **map** : `Provenance`, the location of the parts of the product in the subject, defined as:

PROVENANCE: `nil` | `slot`(a : ATOM) |
`cons`(`head` : PROVENANCE, `tail` : PROVENANCE)

At a fixed point, a SKA-function is identified with a pair (**less**, **formula**), and each IDENTITY entry corresponds to a SKA-function call with arguments captured in **more**.

Let me now define the partial evaluator $\mathcal{F} : \mathbb{G} \to \mathbb{G}$, that for each **id** : IDENTITY partially evaluates **formula** against **more**, constructing a new **dat** : DATUM for that **id**:

- Partial evaluation starts from the subject itself, tagged with the provenance **slot**(1)—the subject sits at axis 1 of itself. From there every intermediate value carries a PROVENANCE that says which part of the subject it was taken from, or `nil` if it was not taken from the subject at all;
- The Nock formula is partially evaluated against the annotated subject:
  - In cons case `[[b c] d]` head and tail are evaluated and the products are consed;
  - Nock 0 produces the appropriate axis from the annotated subject, if present;
  - Nock 1 produces a fully known sock with `nil` provenance;
  - In Nock 3, 4, 5, 12 child formulas are evaluated and an empty product is produced;
  - Nock 6 produces an intersection of values of both branches: the results are intersected on both SOCK values and the provenances;

- In Nock 7 the second child formula is evaluated against the product of the evaluation of the first child against the subject;
- In Nock 11 the product of the hinted formula is produced, and in a dynamic case `[a 11 [b c] d]` the hint-formula is evaluated and its product is ignored;
- Nock 8 is desugared: `[8 p q]` → `[7 [p 0 1] q]`;
- Nock 9 is desugared: `[9 p q]` → `[2 [0 p] q]`;
- Finally, in Nock 2 the formula-formula is evaluated, and if its product is not known the call is considered to be indirect, and an empty product is returned. Otherwise, the subject of the callee is calculated and the appropriate Datum is obtained: $\mathbb{G}(\texttt{calleeSubject}, \textbf{formula})$. The code usage mask of the current call is updated using the provenance of the callee's formula noun and the provenance of the callee's subject plus the code usage mask of the callee. Details like recursion detection are discussed in "Recursion detection" below. The product is the Sock from the callee's Datum, with the provenance composed with the subject's provenance.

Finally, the call graph is constructed by finding `fix` $\mathcal{F}$: we build a chain $[\mathbb{G}_\perp, \mathcal{F}(\mathbb{G}_\perp), \mathcal{F}(\mathcal{F}(\mathbb{G}_\perp)), \ldots]$ until it converges. $\mathbb{G}_\perp$ is a mapping $\mathbb{G}$ such that for every $\mathbf{id}$: Identity, $\mathbb{G}_\perp(\mathbf{id}) = \text{Datum}_\perp$, and $\text{Datum}_\perp$ is the minimal Datum for a function call: an unknown result with no provenance and empty **less**, as we assume no code is used.

2.3 Correctness and termination guarantees

The final value of the chain above is evidently `fix` $\mathcal{F}$. What I would like to demonstrate, if not fully prove, is that the chain is finite and the resulting fixed point is almost always the least fixed point of $\mathcal{F}$: the code usage masks capture no more than the parts of the subject that could actually be used as code,

except for some unusual recursive cases, discussed later (see the section "Homeomorphic embedding").

2.3.1 Kleene iteration

Let me introduce some more vocabulary. A *partially ordered set* is a set with an ordering $\leq$ defined such that, for certain pairs of elements of the set, one precedes the other. A *lattice* is a partially ordered set that for each pair of elements has a *join* $a \vee b$ and a *meet* $a \wedge b$ such that:

$$a \wedge b \leq a \leq a \vee b \text{ and } a \wedge b \leq b \leq a \vee b$$

A *complete lattice* is a lattice that has a join and a meet for any subset of the lattice.

It can be readily seen that, for a given noun N, the set of all socks generated by masking data away from N forms a complete lattice with an ordering $\underset{\mathbf{sock}(N)}{\leq}$ such that the fully unknown sock $\bot_{\mathbf{Sock}}$ is the *bottom element* of the lattice (i.e. it is the infimum of the entire set), and the fully known sock $\top^N_{\mathbf{Sock}}$ is the *top element* (the supremum of the set). The ordering then tells us whether two socks nest: $A \underset{\mathbf{sock}(N)}{\leq} B$ if A and B do not have data that contradicts N and if B has at least as much data as A.

The Kleene fixed-point theorem (P. Cousot and R. Cousot, 1979) states that $\sup\left([\mathbb{G}_\bot,\ \mathcal{F}(\mathbb{G}_\bot),\ \mathcal{F}(\mathcal{F}(\mathbb{G}_\bot)), \ldots]\right)$ is `lfp` $\mathcal{F}$, or *the least fixed point* of $\mathcal{F}$, as long as $\mathcal{F}$ is monotonic: for any two mappings a and b, $a \leq b \Rightarrow \mathcal{F}(a) \leq \mathcal{F}(b)$.

If $\mathbb{G}$ could be represented as a simple product of socks, and if the chain was guaranteed to be finite, then the iterative process described above, known as *Kleene iteration*, would obviously produce `lfp` $\mathcal{F}$, as $\mathbb{G}$ would also form a complete lattice, and $\mathcal{F}$ only adds information on each iteration, never subtracting anything.

2.3.2 Recursion detection

To make sure that the iteration chain is actually finite, we need to be able to detect recursive calls to add pessimizations to them. Without that, a simple tail-recursive Nock expression that produces a list of nouns could be reevaluated on each fixed-point iteration because its product changed. Simple recursion like in that example is detected as follows: for a given Nock 2 eval, if its subject nests under the subject of one of its transitive callers with the same formula, we assume that the Nock 2 call in question is a recursive call to that transitive caller, and its DATUM is returned to the caller with the product replaced with a fully unknown sock with no provenance.

Masking the product this way keeps the chain finite, but it costs us the strict monotonicity of $\mathcal{F}$ over the subjects and results of SKA-functions: a call that one iteration treats as recursive may, on a later iteration, no longer satisfy the recursion condition, at which point it returns $\text{DATUM}_{\perp}$ instead—a strictly smaller value that shrinks the recorded code usage rather than growing it. However, once the transitive caller is no longer classified as a recursion target, it is removed from the finite set of potential recursion targets, so such non-monotone steps can occur only finitely many times and the iteration still converges.

2.3.3 Cons denormalization

Another avenue for infinite Kleene chains is the dynamic generation of Nock code to `eval`. Since Nock 3, 4, and 5 return unknown results, the only way to synthesize a new formula is to cons existing ones together.

The prevention strategy is simple: a consed noun as a whole may never be used as a formula—only the components that were consed into it may. This bounds the executable formulas to the finitely many subtrees already present in the subject-formula pair, making the set of possible SKA-functions it generates finite.

2.3.4 Homeomorphic embedding

Finally, another way to produce infinite Kleene chains is to cons together the subject, not the formula, to produce new SKA-function candidates. This example in Hoon is demonstrative:

```
=/  t  |.(0)
|-  ^-  ~
?:  =(3 $:t)  ~
$(t |.(+($:t)))
```

Here `t` is a trap that accumulates previous values of `t` in its payload. Each recursive iteration generates a new subject for the `$:t` eval in the conditional, infinitely expanding the call stack.

To prevent this, when two calls are checked for simple recursion (that is, whether their subjects straightforwardly nest) and turn out not to be simply recursive, we also check whether the caller's subject is homeomorphically embedded into the callee's (a termination criterion borrowed from online partial evaluation (Leuschel, 2002)): caller $\trianglelefteq$ callee, defined as follows:

- For every Sock a : $a \trianglelefteq a$.
- *Coupling rule*: if a and b are Sock cells and $\text{Head}(a) \trianglelefteq \text{Head}(b)$ and $\text{Tail}(a) \trianglelefteq \text{Tail}(b)$, then $a \trianglelefteq b$.
- *Diving rule*: if b is a Sock cell and $a \trianglelefteq \text{Head}(b)$ or $a \trianglelefteq \text{Tail}(b)$, then $a \trianglelefteq b$.

When homeomorphic embedding is detected, the callee's subject is replaced with the most specific generalization (MSG) of the two subjects—the most specific sock that both subjects nest under, which keeps the data where the two agree and masks it out where they differ—effectively erasing the accumulating part.

Kruskal's tree theorem (Kruskal, 1960) guarantees exactly this termination: the finite trees over a finite set of labels are well-quasi-ordered by homeomorphic embedding, so any infinite sequence of them must contain an earlier tree that is homeomorphically embedded into a later one. The bound this gives is enormous, though: even a handful of labels admits

embedding-free sequences of astronomical length, the classic illustration being TREE(3), a number too large to meaningfully describe. I did try to construct subject-formula pairs whose embedding-free sequences grow even exponentially in the size of the pair, and could not—in every attempt the accumulating part was either masked out by the recursion product or collapsed by the very restrictive Nock 6 intersection. So it appears that worst-case chains are at most linear in length relative to the size of the subject-formula pair.

2.4 Optimizations

2.4.1 Worklist

While the formal definition of $\mathcal{F}$ operates on an infinite mapping $\mathbb{G}$ and partially evaluates every member of the mapping, in reality the call graph is finite. Further, in each fixed-point iteration we only need to evaluate function calls whose immediate callees' DATUM entries have changed since the last iteration, otherwise the evaluation would not give new results. This is addressed with a worklist: a set of IDENTITY entries that includes brand new calls, not present in the previous value of $\mathbb{G}$, and calls whose callees changed since the last iteration. This fixed-point computation algorithm falls into the family of chaotic fixed-point iterations (Geser et al., 1994), and it also produces `lfp` $\mathcal{F}$.

2.4.2 Memoization within a fixed-point iteration

To save time within one fixed-point iteration, the result of a partial evaluation of a function call was saved into an iteration-local cache. Before evaluating a function call the cache was checked, and on a hit the saved result was returned. Extra care was applied to prevent the caching from pessimizing calls with transitive indirect callees. Firstly, if a function captured some part of its subject in the product, and some part of the captured subject subtree was unknown, the function was not memoized to allow analysis of more specific calls. Secondly, whenever an indirect call was performed, the provenance of

the formula was recorded, and during cache look-ups the algorithm checked whether the cache candidate had known data at the places where the cached call tried to obtain a formula for its indirect call. If there was any known data the cache entry was skipped, again allowing more specific calls than the cached one to be analyzed.

There were also some optimizations that ultimately were not included in the algorithm because the machinery to support them took more time than the optimizations saved. Let me briefly mention them.

2.4.3 Memoization of the finalized results

Whenever a function call and all of its transitive callees were no longer in the worklist we could consider the function to be finalized: no other updates to the call graph could cause reanalysis of such functions. To support such memoization I had to keep track of the transitive closure of the call graph (i.e. the graph whose edges connect a function call with any function call reachable down the stack, not only the immediate callees).

2.4.4 Transitive closure of the reversed call graph for faster recursion detection

Instead of walking the reversed call graph to find a candidate for a recursive call I tried keeping track of the transitive closure of the reversed call graph. Incremental updates to that graph and to the transitive closure of the direct call graph mentioned above worked in the same way:

- Firstly, the set of vertices whose immediate children changed was collected, called a *seed*;
- Then the appropriate reversed graph was walked, assembling a set of vertices that could reach the seed set, or *affected* set;
- The reversed subgraph of affected vertices was assembled;

- The reversed subgraph was condensed with Tarjan's SCC algorithm; the SCCs of the reversed subgraph were toposorted, i.e. predecessors (in the reversed graph) first;

- For each SCC a *closure* was computed, and each member of the SCC got edges to every member of the closure;

- The *closure* was computed by taking a union over every immediate child of every member of the SCC: $\{\texttt{child}\}$ if the child was in the SCC, else $\{\texttt{child}\} \cup \texttt{TC}\,[\texttt{child}]$, where `TC` is the previous value of the transitive closure.

This approach allowed me to avoid fixed-point iterations in the construction of the transitive closures, but it was still not worth it for all tested workloads.

3 Compilation

Once we have the call graph, we can start compiling bodies of SKA-functions into a static single-assignment (SSA) intermediate representation (IR)—that is, compiling the tree-shaped Nock formula of a function into a stream of instructions that assign parts of the input subject and products of computations to immutable variables, each written exactly once. That IR would then be subject to further optimization and compilation passes; SSA form was chosen to simplify these subsequent passes.

During the call graph construction phase we made no assumptions about the shape of the input subject, nor about the shape of the resulting noun. The simplest way to treat this in the compilation phase would be to assume that all functions take a single noun with an unknown shape as an input and return a noun with an unknown shape as a result. The problem with this approach becomes apparent when we consider how an n-ary gate is typically called and evaluated in Nock, as in the current bytecode Nock interpreter in Vere:

- n input arguments from different parts of the subject and/or Nock 1 formulas are consed into a single noun tuple;

- The gate's arm is evaluated to produce a core with the default sample;
- The sample is edited with the tuple;
- The $ arm is evaluated: we enter the callee's body;
- In the callee's code, the argument tuple is split back into n pieces, and computations are run with them.

All of this extra work could have been avoided if we knew ahead of time that a given function uses data from certain axes, for example, that a binary function uses data at axes 12 and 13. That way, when compiling the caller of that function, we could just provide the variables that hold the inputs for the callee, and when compiling the callee, we wouldn't have to emit subject decomposition code to get the input arguments.

We can figure out the data requirements of a function by observing which parts of the input subject the function tries to access with Nock 0, and requiring the presence of data at an axis whenever the function would have crashed had that axis been missing.

So a SKA-function behind this Hoon gate would be binary with arguments at axes 12 and 13:

```
|=  [a=@ b=@]
(add b (mul 10 a))
```

because `a` and `b` are accessed individually and unconditionally, and the function would have crashed, were it called with an atom as its sample. This function, on the other hand, would be unary:

```
|=  [a=@ b=@]
(add 42 (add +<))
```

because here the entire sample is given to `+add`. If that gate was called with an atom as its sample, the crash would have happened inside `+add`, which we would have to call to preserve stack trace correctness.

There are some complications when it comes to obtaining data requirements, solutions for which will be described below. Firstly, when a caller and a callee belong to the same SCC,

compiling them both, and thus finding their data requirements, needs to be done simultaneously.

Secondly, the data usage of a Nock formula cannot be simply composed out of its sub-formulas, when iterating over them in one direction or the other, due to branches. Consider this Hoon example:

```
++  maybe-add-to-42
  |=  delta=(unit @)
  ^-  @
  ?~  delta  42
  (add 42 u.delta)
```

Here the desired argument usage of the function represented by this gate would, of course, be "noun at axis 6", which corresponds to `delta`. Let's iterate over the underlying Nock formula in two directions, forwards and backwards, and see what conclusion each direction leads to. When going forward, as is natural for symbolic or actual interpretation:

- The conditional reaches for axis 6, where `delta` is located;
- In the first branch, `42` is returned, and no other axis is reached;
- In the second branch, axis 13 is reached to get `u.delta`, then a direct call to `+add` is performed, which uses no axes;
- When joining the data usage of the branches, we need to take into account that the first branch did not access axis 13, so that axis might not be present in the subject, as would be the case here. We did already reach axis 6 in the conditional, so we can get the data usage of the two by computing the MSG—the most specific generalization, introduced in the section "Homeomorphic embedding"—of the unions of the data usage before the branch with the data usage in each branch, which would give us "noun at axis 6".

If we iterate over the formula backwards, as is customary for destination-driven code generation (Dybvig, Hieb, and Butler, 1990) as discussed below, we obtain a different result:

- The first branch uses nothing from its subject;
- The second branch accesses axis 13;
- Their MSG is axis 1—in the first branch nothing was asserted about the shape of the subject, so in the second branch we would have to add subject decomposition code;
- The conditional accesses axis 6;
- The body overall needs a union of usages "axis 6" and "axis 1", which, depending on the representation of the argument split, could devolve to just "axis 1".

Special-casing the conditional would not help in the general case: to properly calculate the data requirements of a branch we need to take an MSG of the unions of the branches' data requirements with the data requirements before and after the branch. Furthermore, this accumulation of lazy data requirements and their final collapse into a single data requirement needs to be done recursively for nested branches as well.

Finally, it is not sufficient to reproduce whether a crash *would* happen if a function was called with an ill-fitting subject; we also need to reproduce the location of the crash, or rather, the state of the stack trace at the moment it occurs. Indeed, `+mink` allows us to materialize the stack trace as a product of Nock evaluation, which demands that we uphold these semantics. Simply ignoring `%mean`/`%spot` hints would mean that calling a binary function with an atom for a sample would *relocate* the crash to the callsite, but to preserve `+mink` semantics we would need to know what the stack trace would look like in the callee at the place where the crash would occur in Nock.

Knowing the shape, or more generally, the type of the product of a function would also enable some optimizations. Cell-checking code could be eliminated, for example, if we knew that a given function always returns atoms. For simplicity,

however, this kind of analysis is not yet performed, and all functions are assumed to return nouns of unknown shape.

3.1 Fixed-point loop for SCCs

I resolve the circularity of compiling functions in an SCC by compiling an entire SCC at a time, with a starting assumption that all functions are nullary, i.e. they use no data from their subjects. A worklist algorithm similar to the one used in the call graph construction is applied, where the initial worklist contains the entire SCC, and members are re-queued if the data requirements of their immediate callees changed. On subsequent iterations the data requirements are joined with the previous value by taking their MSG and emitting code to deconstruct the pessimized parts; this is necessary to prevent divergence of the data requirements. On achieving the fixed point the entire '(map bell straight)' is returned, where `$bell` is the SKA-function identifier and `straight` contains the SSA IR.

On direct calls to jetted functions the supplied data usage is used—if a jet driver exists then the data requirements of a function are surely known. On calls within the current SCC the latest best guess is used. On calls outside of the current SCC the algorithm is invoked recursively.

3.2 (Lazy) destination-driven code generation

The compilation follows the destination-driven code generation (DDCG) style (Dybvig, Hieb, and Butler, 1990): the body of the SKA-function is walked backwards or, in the case of tree-shaped Nock formulas, iterated from the inner subformulas outwards. Every Nock formula consumes a given `$goal`, describing what must happen to its product, and produces a data requirement that must be satisfied by the preceding code, or by the input arguments in the case of the top-level formula. The data requirement is described by the `$need-lazy` type in Listing 1.

A `$goal`, in turn, is one of three shapes (Listing 2).

The generated code is organized into *basic blocks*: a basic block contains a list of input arguments—always empty

Listing 1: The `$need-lazy` type providing the data requirement for Nock formula code generation.

```
::  @uwoo - basic block index
::  @uvre - SSA register index
+$  need
  $~  [%none ~]
  ::  cons case: something is needed from the head
  ::  and/or the tail
  $^  [p=need q=need]
  $%  ::  nothing is needed from this noun
      [%none ~]
      ::  this noun needs to be in `r`
      [%this r=@uvre]
      ::  both the entire noun is needed in `r` and
      ::  something from its head and/or tail is
      ::  also needed.
      ::  if `c`: the downstream code asserts that
      ::  `r` contains a cell
      [%both r=@uvre c=? h=need t=need]
  ==
+$  need-lazy
  $+  need-lazy
  ::  recursive type
  $;  |-
  $:  ::  data requirements on this level of branching/
      ::  stacktrace hints
      sure=need
      ::  data requirements of branches, with their
      ::  basic block indices
      fork=(list [y=[o=@uwoo laz=$] n=[o=@uwoo laz=$]])
      ::  data requirements of blocks guarded with a
      ::  stacktrace-affecting hint, with their basic
      ::  block indices
      bond=(list [o=@uwoo laz=$])
  ==
```

Listing 2: The `$goal` type providing the objective for Nock formula code generation.

```
::  $jmp: call given basic block with the arguments.
::  Arguments replace phi nodes
+$  jmp  [args=(list @uvre) there=@uwoo]
+$  goal
  $%  ::  product is used as a conditional
      [%pick z=jmp o=jmp]
      ::  tail position
      [%done ~]
      ::  put the product into registers of `laz`,
      ::  then go to `then`
      [%next laz=need-lazy then=jmp]
  ==
```

for blocks with a single predecessor—a straight-line list of SSA operations, and a control-flow operation that terminates the block. Basic block parameters were chosen instead of phi-nodes (the traditional SSA device for merging values that arrive from different branches), similar to the Cranelift IR approach (Bytecode Alliance, 2024), to simplify codegen. At a merge point of two branches, the common successor block is "called" by the final blocks of the branches with the appropriate arguments.

The starting goal is `[%done ~]`, as we are in the tail position. This goal enables tail-call optimization.

Needs produced by consecutive computations, as in autocons, are unified into one by concatenating the lazy lists and emitting the code needed to copy nouns between registers. Direct calls produce needs by checking the hot state table, by using the latest best guess if the callee is in the current SCC, or by recursively entering the compilation of the child SCC. When a lazy need must be satisfied with a noun—either a Nock 1 literal or a Nock 2 or Nock 12 product—the lazy need is collapsed by walking the recursive data structure and emitting deconstructing code into the basic blocks attached to the nested lazy needs. Similarly, needs satisfied with Nock 3/4/5 that produce

an atom are collapsed by emitting assertion code into the contained basic blocks. Hints with crash-relocation boundaries (`%mean` and `%spot` for Nock virtualization correctness, `%slog` for better UX) wrap the need by including it in the `bond` lazy list.

Once the top-level formula's lazy need is produced, it is collapsed into a simple `$need` by walking the nested lists inwards, propagating the top-level guaranteed argument usage, then collapsing the lists outwards. For branches, the MSG of the yes- and no-branch requirements is constructed, with appropriate deconstructing code emitted into the relevant basic blocks before branches and crash relocation boundaries. Once the need is collapsed, a register-rewrite pass renumbers the registers so that the input registers are numbered 0 to n, simplifying the calling convention. The final compilation product for a SKA-function is then a register-less need recording the shape of the argument tree, an argument count, and a `(map @uwoo blob)`. Index 0 serves as the function's entry point.

3.3 Two modes of compilation

To correctly compile a function that would crash if called with a subject of an incorrect shape, while also trying not to pessimize its compilation by making defensive assumptions, a function can be compiled twice: the first time to get the shape of its subject, and the second time to compile it for the subject as a single noun of an unknown shape. To prevent the call graph from partitioning into two disconnected parts, each function call in the pessimized version is wrapped with cell checks. That way the caller function tries to disassemble the arguments itself at the callsite, and if any of the cell checks fail, it falls back to calling the pessimized version of the callee. Thus the pessimized partition of the call graph will call into the optimized partition whenever possible, reducing the time the interpreter spends in the pessimized partition.

References

Bytecode Alliance (2024). *Cranelift IR Reference*. Accessed: 2026-07-22. URL: https://github.com/bytecodealliance/wasmtime/blob/2c72afc5ccd3fa385d83c6b144a3d305d94550d0/cranelift/docs/ir.md.

Cousot, Patrick and Radhia Cousot (1979). "Constructive Versions of Tarski's Fixed Point Theorems." In: *Pacific Journal of Mathematics* 82.1, pp. 43–57. DOI: 10.2140/pjm.1979.82.43. URL: https://www.di.ens.fr/~cousot/COUSOTpapers/Tarski-79.shtml.

~dozreg-toplud, K. Afonin (2026). "Subject Knowledge Analysis." In: *Urbit Systems Technical Journal* 3.1. URL: https://ustj.urbit.org/article/v03-i01/subject-knowledge-analysis.

Dybvig, R. Kent, Robert Hieb, and Tom Butler (1990). *Destination-Driven Code Generation*. Tech. rep. 302. Indiana University Computer Science Department. URL: https://bernsteinbear.com/assets/img/ddcg.pdf.

Geser, Alfons et al. (1994). *Chaotic Fixed Point Iterations*. Tech. rep. MIP-9403. Passau, Germany: Universität Passau, Fakultät für Mathematik und Informatik. URL: https://www.swt-bamberg.de/luettgen/publications/pdf/Passau-MIP-9403.pdf.

Kruskal, Joseph B. (1960). "Well-Quasi-Ordering, the Tree Theorem, and Vázsonyi's Conjecture." In: *Transactions of the American Mathematical Society* 95.2, pp. 210–225. DOI: 10.1090/S0002-9947-1960-0111704-1. URL: https://www.ams.org/journals/tran/1960-095-02/S0002-9947-1960-0111704-1/S0002-9947-1960-0111704-1.pdf.

Leuschel, Michael (2002). "Homeomorphic Embedding for Online Termination of Symbolic Methods." In: *The Essence of Computation: Complexity, Analysis, Transformation*. Vol. 2566. Lecture Notes in Computer Science. Springer,

pp. 379–403. URL:
https://eprints.soton.ac.uk/257252/.
~sorreg-namtyv, Curtis Yarvin (2013) “Nock 4K”. URL:
https://docs.urbit.org/language/nock/reference/definition (visited on ~2024.2.20).